

INGERIERIA EN ELECTRONICA Y AUTOMATIZACION

MATERIA: FUNDAMENTOS DE PROGRAMACIÓN

TEMA: 20 EJERCICIOS

NOMBRE: TEJADA SEBASTIÁN

NRC:36460

FECHA:26/10/2026

- Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

Análisis del problema

Entrada: Números enteros x, z

Proceso: Sumar x y z para luego dividirlo para dos

Salida: La media de los dos números es:

Restricciones: Solo se pueden ingresar números enteros

Tabla de objetos

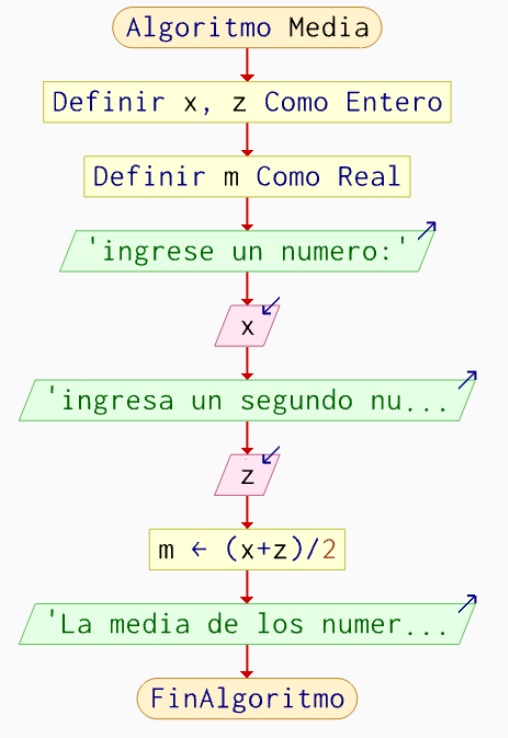
OBJETO	*TIPO*	*NOMBRE*
Numero 1	Real	X
Numero 2	Real	Z
Media	real	m

Seudocódigo

```
Algoritmo Media
        Definir x,z, Como Entero;
          Definir m Como Real
        Escribir "ingrese un numero:";
        Leer  x;
        Escribir  "ingresa un segundo numero:";
        Leer z;
        m<- (x+z)/2;
        Escribir "La media de los numeros es",m;

Fin Algoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	NUM1	NUM2	CALCULO	SALIDA
1	4	-	-	-
2	4	2	-	-
3	4	2	(NUM1+NUM2)/2	3

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 1 media
 void main()
{

    int x,z,m;

    printf("Ingrese primer numero: ");
    scanf("%d",&x);
    printf("Ingrese segundo numero: ");
    scanf("%d",&z);

    m=(x+z)/2 ;
    printf("La media es:%d\n",m);


}
```

```
"C:\Users\sabes\OneDrive\De
Ingrese primer numero: 4
Ingrese segundo numero: 2
La media es:3

Process returned 14 (0xE)   execution time : 3.579 s
Press any key to continue.
```

- Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

Análisis del problema

Entrada: Un numero entero x

Proceso: Calcula el valor absoluto de x y elévalo al cubo

Salida: El valor absoluto de x elevado al cubo

Restricciones: El numero debe ser entero

Tabla de objetos

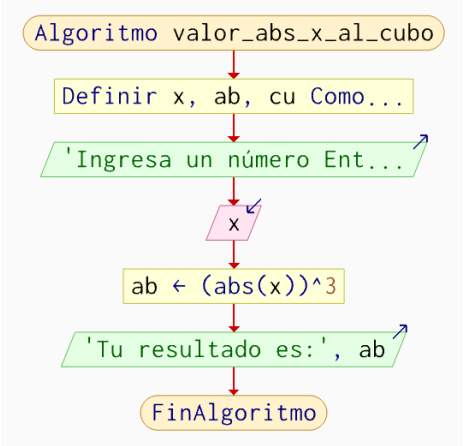
OBJETO	*TIPO*	*NOMBRE*
Numero	Entero	X
Calculo	Entero	Ab

Seudocódigo

```
Algoritmo valor_abs_x_al_cubo
    Definir x,ab Como Entero;
    Escribir "Ingresa un número Entero";
    Leer x;
    ab<-(abs(x))^3;
    Escribir "Tu resultado es:",ab;

FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	X	ab	RESULTADO	SALIDA
1	-5	abs(-5))^3	125	“Tu resultado es 125”
2	3	abs(3))^3	27	“Tu resultado es 27”
3	45	abs(45))^3	91125	“Tu resultado es 91125”

Ejecutado Codeblocks

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  //Ejercicio 2 valor abs x
4  void main()
5  {
6      int x,ab;
7
8      printf("Ingrese numero: ");
9      scanf("%d",&x);
10     ab=abs(x)*abs(x)*abs(x);
11     printf("Tu resultado es: %d\n",ab);
12
13
14 }
15
```

```
"C:\Users\sabes\OneDrive\De
Ingrese numero: -5
Tu resultado es: 125

Process returned 21 (0x15)   execution time : 3.244 s
Press any key to continue.
```

- Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

Análisis del problema

Entrada: Se ingresará dos números enteros

Proceso: Se analizará si el primer número es divisible para el segundo número.

Salida: Se mostrará a pantalla si el primer numero es divisible para el segundo

Restricciones: El segundo número no pude ser 0 porque la división para 0 no existe.

Tabla de objetos

OBJETO	TIPO	NOMBRE	VALOR
NUMERO 1	ENTERO	Num1	8
NUMERO 2	ENTERO	Num2	2

Seudocódigo

```
Algoritmo Divisible
        Definir num1, num2 Como Entero;
        Escribir "Ingrese el primer número (dividendo):";
        Leer num1;
        Escribir "Ingrese el segundo número (divisor):";
        Leer num2;

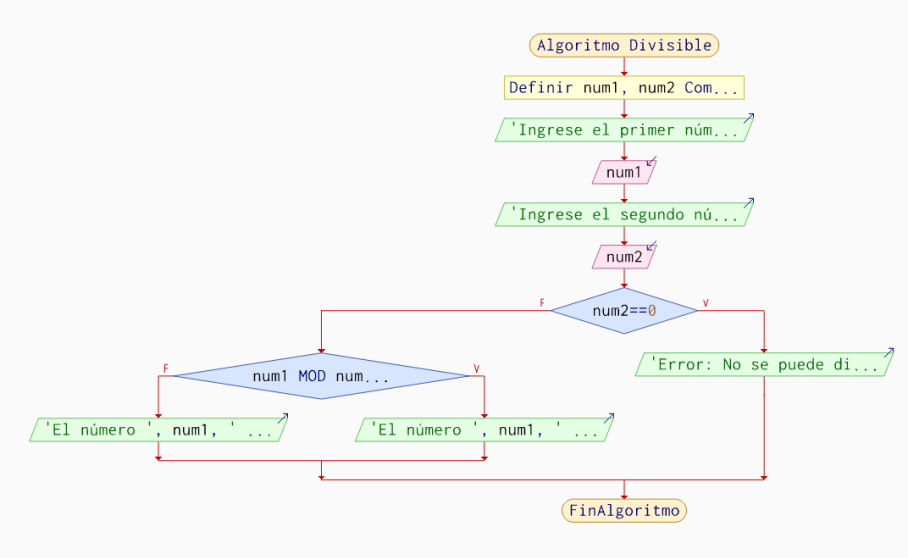
        Si num2 == 0 Entonces
            Escribir "Error: No se puede dividir entre cero.";
        Sino
            Si num1 % num2 == 0 Entonces
                Escribir "El número ", num1, " SÍ es divisible entre ", num2;
            Sino
                Escribir "El número ", num1, " NO es divisible entre ", num2;
```

Fin Si

Fin Si

Fin Algoritmo

Diagrama de flujo



Prueba de escritorio

PASO	OPERACIÓN	NUM1	NUM2	SALIDA
1	LEER NUM1	8	-	-
2	LEER NUM2	8	2	-
3	Calcular n1%n2	8	2	-
4	n1%n2==0	8	2	Su número si es divisible

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 3, Divisible, Tejada

void main()
{
    int n1,n2;

    printf("Ingrese numero 1: ");
    scanf("%d",&n1);
    printf("Ingrese numero 2: ");
    scanf("%d",&n2);

    if (n2==0){
        printf("Error no se puede divir para 0");
        printf("\n");
        printf("Vuelve a ingresar");
    }
    else if(n1%n2==0){

        printf("EL numero %d\n",n1);
        printf("\n");
        printf("Si es divisible para %d\n",n2);
    }
    else{
        printf("EL numero %d\n",n1);
        printf("\n");
        printf("No es divisible entre %d\n",n2);
    }
}
```

```
"C:\Users\sabes\OneDrive\De
Ingrese numero 1: 8
Ingrese numero 2: 2
EL numero 8

Si es divisible para 2

Process returned 23 (0x17)   execution time : 4.359 s
Press any key to continue.
```

- Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

Análisis del problema

Entrada: Numero x entero

Proceso: Se determinará si el número pertenece dentro del intervalo o si en dicho caso está fuera de él.

Salida: Se presentará un mensaje de el numero se encuentra dentro del rango, si el numero esta entre 0-10 y un mensaje de no pertenece si dicho numero no está dentro del rango.

Restricciones:

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Primer numero	Entero	X

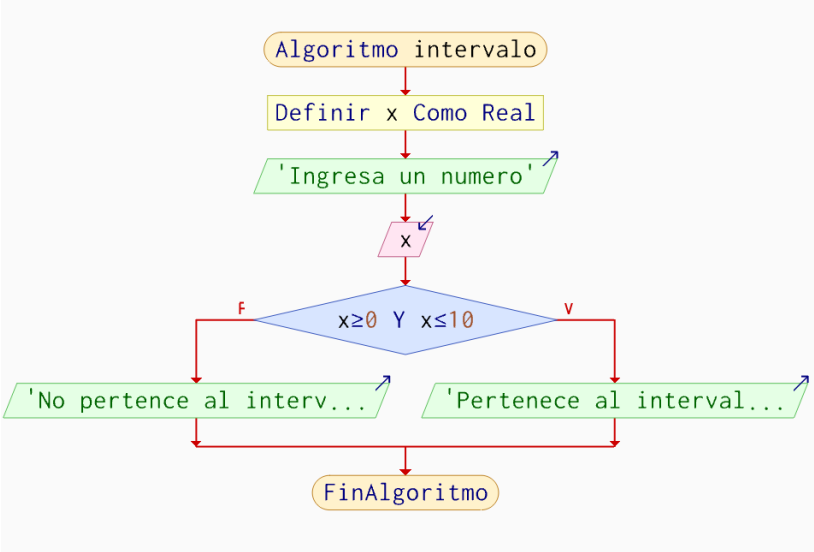
Seudocódigo

Algoritmo intervalo

Definir x como real;

```
    Escribir "Ingresa un numero";
    Leer x;
    si x>=0 && x<=10 Entonces
        Escribir "Pertenece al intervalo entre 0-10";
    SiNo
        Escribir "No pertence al intervalo 0-10";
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	NUM1	INTERVALO 0-10	SALIDA
1	5	VERDADERO	“Si pertenece al intervalo”
2	-1	FALSO	“No pertenece al intervalo”
3	10	VERDADERO	“Si pertenece al intervalo”
4	15	FALSO	“No pertenece al intervalo”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 4 intervalo
void main()
{
    int x;

    printf("Ingrese numero: ");
    scanf("%d",&x);
    if (x>=0&&x<=10){
        printf("\n");
        printf("%d Si pertenece al intervalo de 0-10",x);

    }
     else{
        printf("\n");
        printf("%d No pertenece al intervalo de 0-10",x);

     }

    }
```

```
"C:\Users\sabes\OneDrive\De
Ingrese numero: 5

5 Si pertenece al intervalo de 0-10
Process returned 35 (0x23)   execution time : 2.492 s
Press any key to continue.
```

- Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa. Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %). Sea c el número de segundos a convertir. Como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos, el cociente entero c / (24 · 60 · 60) es el número de días completos contenidos en c. Asimismo, c % (24 · 60 · 60) es el número de segundos restantes tras descontar esos días. Se procede de igual manera para obtener las horas (1 hora = 60 · 60 segundos) y los minutos (1 minuto = 60 segundos).

Análisis del problema

Entrada: Un número c positivo que sea el valor de los segundos que queremos transformar

Proceso: Calcula los días, horas y minutos, segundos utilizando la división entera y el resto de la división.

Salida: Se mostrará los días, minutos y segundos que corresponde al número c ingresado

Restricciones: El valor del numero c debe ser positivo

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Valor de los segundos	Entero	c
Operación de Dias	Entero	dias
Operación de Horas	Entero	horas
Operación de segundos	Entero	segund
Valor del resto	Entero	resto

Seudocódigo

```
Algoritmo Unidades_de_tiempo
        Definir c, dias, horas, minutos, segund, resto Como Entero;
        Escribir "Ingresa el número de segundos:";
        Leer c;
        // Días completos
        dias <-trunc( c / 86400);
        resto <- c % 86400;
        // Horas completas
        horas <-trunc( resto / 3600);
        resto <- resto % 3600;
        // Minutos completos
        minutos <-trunc( resto / 60);
        segund <- resto % 60;
        Escribir "Días: ", dias;
        Escribir "Horas: ", horas;
        Escribir "Minutos: ", minutos;
        Escribir "Segundos: ", segund;
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	C	dias	horas	minutos	segund	SALIDA
1						“días:” “horas:” “minutos:” “segundos:”
2						“días:” “horas:” “minutos:” “segundos:”
3						“días:” “horas:” “minutos:” “segundos:”
4						“días:” “horas:” “minutos:” “segundos:”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 5 conversion tiempo

int main() {
    int c, dias, horas, minutos, segundos, resto;

    printf("Ingresa el numero de segundos del evento: ");
    scanf("%d", &c);

    //dias completos (1 dia = 86400 seg)
    dias  = c / 86400;
    resto = c % 86400;

    //horas completas (1 hora = 3600 seg)
    horas = resto / 3600;
    resto = resto % 3600;

    //Calcular minutos completos (1 minuto = 60 seg)
    minutos  = resto / 60;
    segundos = resto % 60;

    printf("\n--- Conversion ---\n");
    printf("Dias:     %d\n", dias);
    printf("Horas:    %d\n", horas);
    printf("Minutos:  %d\n", minutos);
    printf("Segundos: %d\n", segundos);
}
```

```
"C:\Users\sabes\OneDrive\De
Ingresa el numero de segundos del evento: 90

--- Conversion ---
Dias:     0
Horas:    0
Minutos:  1
Segundos: 30

Process returned 0 (0x0)   execution time : 2.535 s
Press any key to continue.
```

- Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

Análisis del problema

Entrada: Introduce valores de alturas positivas

Proceso: Calcula cual de todas las alturas es la máxima, la mínima y la media de todas las alturas ingresadas.

Salida: El valor de la altura máxima, la altura mínima y la media de las alturas

Restricciones: No pueden ser valores negativos

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Valor de altura	Real	Altura
Valor max	Real	Máximo
Valor min	Real	Mínimo
Suma alturas	Real	Suma_alturas

Seudocódigo

Algoritmo Alturas

Definir altura, maximo, minimo, suma_alturas Como Real;

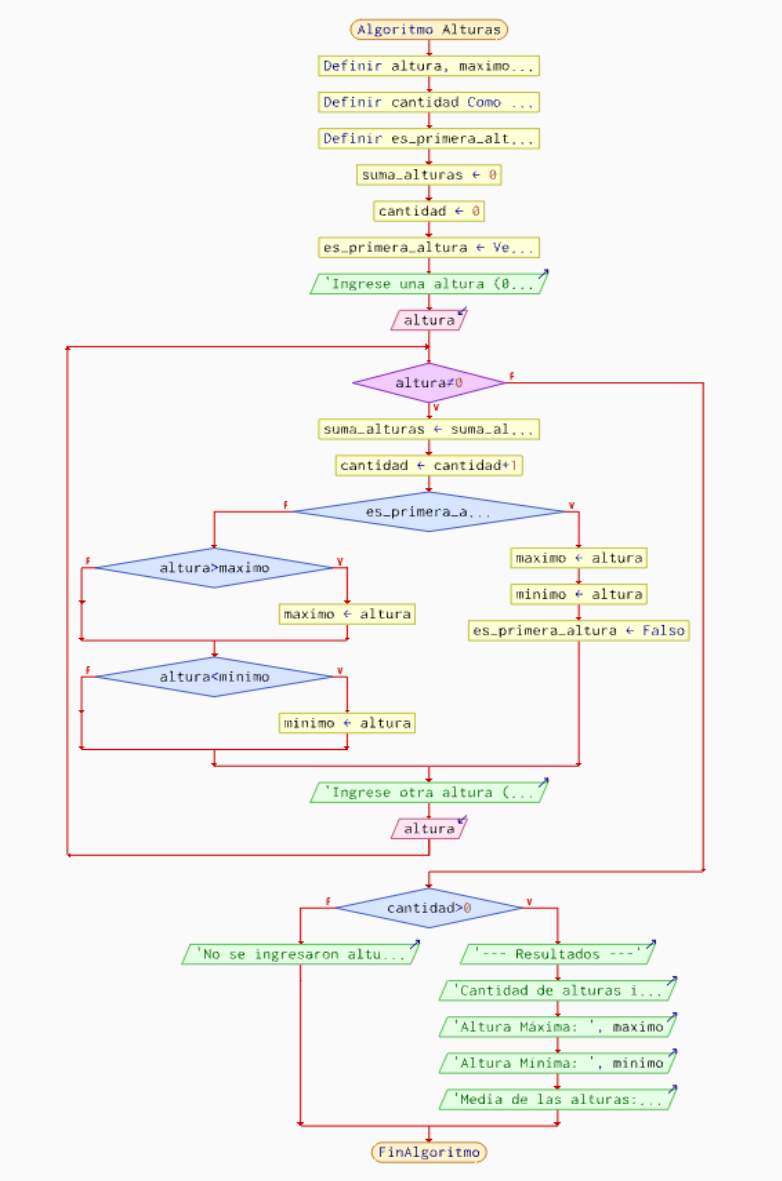
```
Definir cantidad Como Entero;
Definir es_primera_altura Como Logico;
suma_alturas <- 0;
cantidad <- 0;
es_primera_altura <- Verdadero;
Escribir "Ingrese una altura (0 para terminar):";
Leer altura;
Mientras altura <> 0 Hacer
        suma_alturas <- suma_alturas + altura;
        cantidad <- cantidad + 1;
        Si es_primera_altura Entonces
                maximo <- altura;
                minimo <- altura;
                es_primera_altura <- Falso;
        Sino
                Si altura > maximo Entonces
                        maximo <- altura;
                FinSi
                Si altura < minimo Entonces
                        minimo <- altura;
                FinSi
        FinSi
        Escribir "Ingrese otra altura (0 para terminar):";
        Leer altura;
FinMientras

Si cantidad > 0 Entonces
        Escribir "--- Resultados ---";
```

```
            Escribir "Cantidad de alturas ingresadas: ", cantidad;
            Escribir "Altura Máxima: ", maximo;
            Escribir "Altura Mínima: ", minimo;
            Escribir "Media de las alturas: ", suma_alturas / cantidad;
        Sino
            Escribir "No se ingresaron alturas válidas.";
        FinSi

FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	Alturas	max	minimo	Suma de alturas	SALIDA
1	12	-	-	-	-
2	43	43	12	43+12 /2	-
3	65	65	12	43+12+65 /3	-
4	0	65	12	43+12+65/3 =40	“Altura max:” “Altura min:” “Media de alturas:” “Cantidad de alturas:”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 6 alturas
int main() {
    float altura, maximo, minimo, suma_alturas;
    int cantidad;
    int es_primera_altura;

    suma_alturas = 0;
    cantidad = 0;
    es_primera_altura = 1;

    printf("Ingrese una altura (0 para terminar): ");
    scanf("%f", &altura);

    while (altura != 0) {

        suma_alturas = suma_alturas + altura;
        cantidad = cantidad + 1;

        if (es_primera_altura) {
            maximo = altura;
            minimo = altura;
            es_primera_altura = 0;
        } else {
            if (altura > maximo) {
                maximo = altura;
            }

            if (altura < minimo) {
                minimo = altura;
            }
        }

        printf("Ingrese otra altura (0 para terminar): ");
        scanf("%f", &altura);
    }
```

```
"C:\Users\sabes\OneDrive\De
Ingrese una altura (0 para terminar): 12
Ingrese otra altura (0 para terminar): 43
Ingrese otra altura (0 para terminar): 65
Ingrese otra altura (0 para terminar): 0

--- Resultados ---
Cantidad de alturas ingresadas: 3
Altura Maxima: 65.00
Altura Minima: 12.00
Media de las alturas: 40.00

Process returned 0 (0x0)   execution time : 10.701 s
Press any key to continue.
```

- Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero R > 0 tal que $R^2 \leq N$.

Análisis del problema

Entrada: Un numero entero positivo N

Proceso: Calcula la raíz cuadrada entera del número N ingresado

Salida: El valor de la raíz cuadrada entera del número N ingresado

Restricciones: N tiene que ser un número positivo

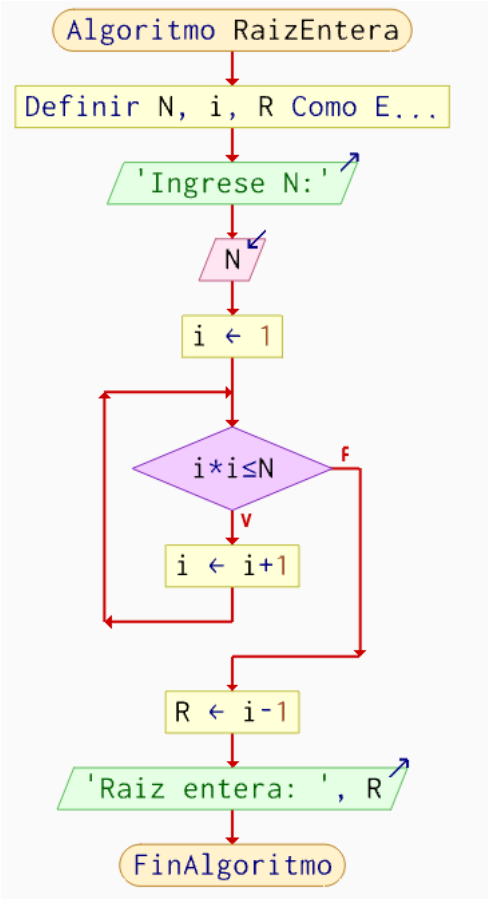
Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero ingresado	Entero	N
Contador	Entero	i
Resultado de la raíz	Entero	R

Seudocódigo

```
Algoritmo RaizEntera
        Definir N, i, R Como Entero;
        Escribir "Ingrese N:";
        Leer N;
        i = 1;
        Mientras i * i <= N Hacer
            i = i + 1;
        FinMientras
        R = i - 1;
        Escribir "Raiz entera: ", R;
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	i	R	SALIDA
1	8	-	-	-
2	8	1^2=1	1	-
3	8	-	2	-
4	8	2^2=4	2	-
5	8	-	2	-
6	8	3^2=9	3	-
7	8	-	2	“Raiz entera es 2”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 7,Raiz entera
void main()
{
    int n,i,r;
    printf("Ingrese un numero : ");
    scanf("%d",&n);
    i=1;
    while (i*i<=n) {
          i=i+1;
    }
    r=i-1;
    printf("Raiz entera: %d\n",r);
}
```

```
"C:\Users\sabes\OneDrive\De
Ingrese un numero : 36
Raiz entera: 6

Process returned 15 (0xF)   execution time
Press any key to continue.
```

- Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

Análisis del problema

Entrada: Un numero positivo mayor a 1

Proceso: Calcula si el número ingresado es un número primo o no

Salida: Nos dice si el número ingresado es primo o no lo es

Restricciones: El número ingresado debe ser positivo y mayor a 1

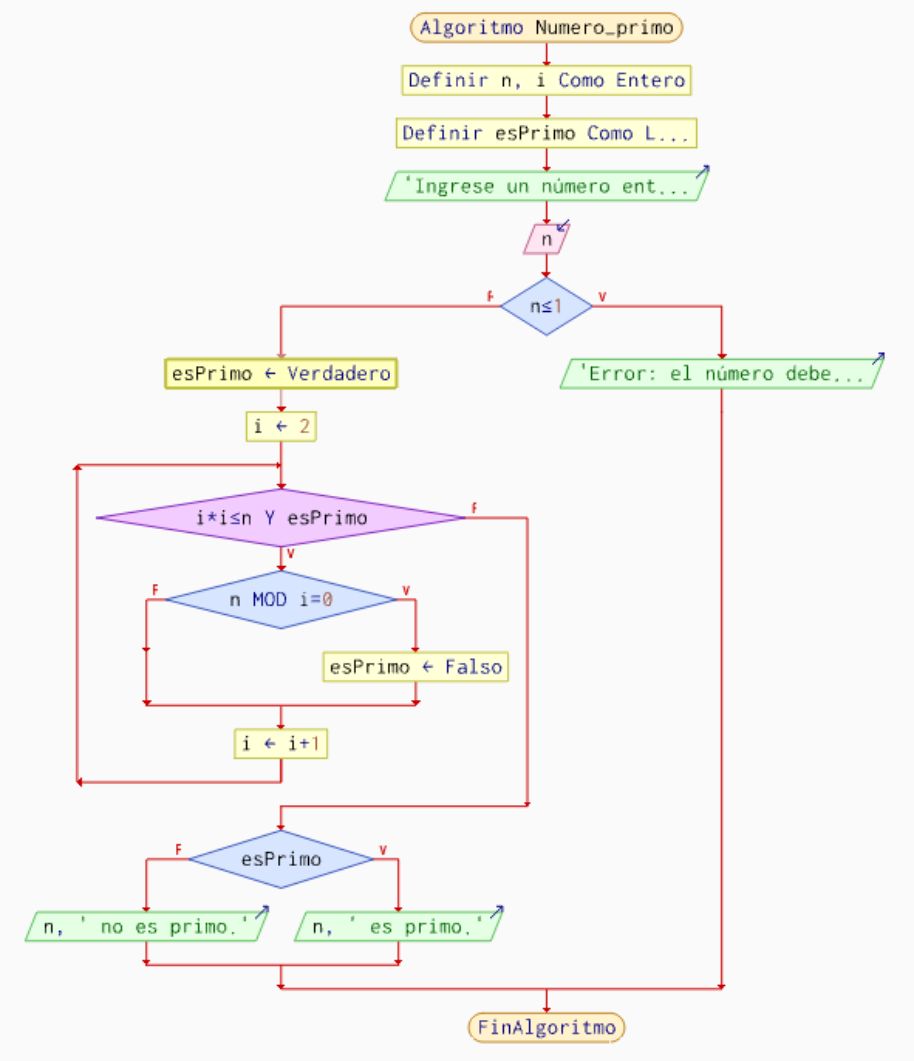
Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero ingresado	Entero	n
Contador	Entero	i
Valor primo	Logico	esPrimo

Seudocódigo

```
Algoritmo Numero_primo
        Definir n, i Como Entero;
        Definir esPrimo Como Logico;
        Escribir "Ingrese un número entero mayor que 1:";
        Leer n;
        Si n <= 1 Entonces
            Escribir "Error: el número debe ser mayor que 1.";
        SiNo
            esPrimo <- Verdadero;
            i <- 2;
            Mientras i * i <= n Y esPrimo Hacer
                Si n % i = 0 Entonces
                    esPrimo <- Falso;
                FinSi
                i <- i + 1;
            FinMientras
            Si esPrimo Entonces
                Escribir n, " es primo.";
            SiNo
                Escribir n, " no es primo.";
            FinSi
        FinSi
FinAlgoritmo
```

Diagrama de flujo

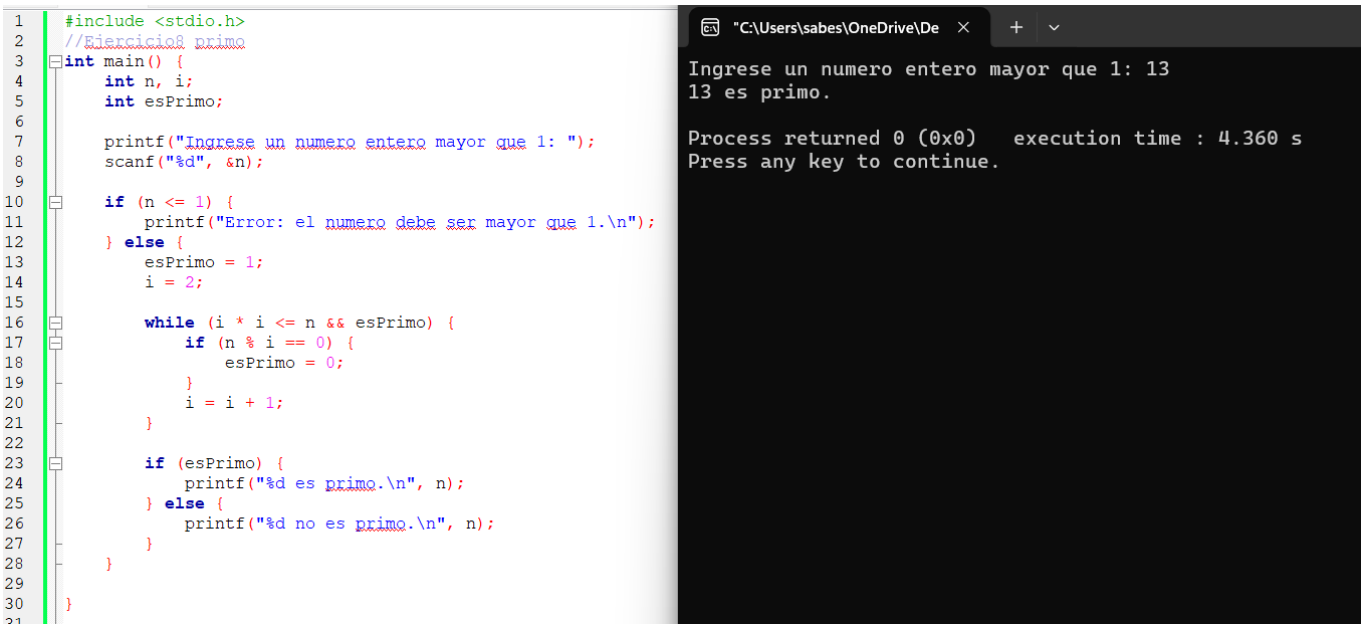


Prueba de escritorio

PASO	n	i	esPrimo	SALIDA
1	7	-	-	-
2	7	-	0	-
3	7	1	0	-
4	7	1	1	-
5	7	2	1	-
6	7	3	1	-
7	7	4	1	-
8	7	5	1	-
9	7	6	1	-

10	7	7	2	-
11	7	8	2	“7 si es primo”

Ejecutado Codeblocks



```c
#include <stdio.h>
//Ejercicio8 primo
int main() {
    int n, i;
    int esPrimo;

    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &n);

    if (n <= 1) {
        printf("Error: el numero debe ser mayor que 1.\n");
    } else {
        esPrimo = 1;
        i = 2;

        while (i * i <= n && esPrimo) {
            if (n % i == 0) {
                esPrimo = 0;
            }
            i = i + 1;
        }

        if (esPrimo) {
            printf("%d es primo.\n", n);
        } else {
            printf("%d no es primo.\n", n);
        }
    }

}
```

```
"C:\Users\sabes\OneDrive\De
Ingrese un numero entero mayor que 1: 13
13 es primo.

Process returned 0 (0x0)   execution time : 4.360 s
Press any key to continue.
```

- Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

Análisis del problema

Entrada: Un intervalo positivo

Proceso: Calcula todos los números primos que se encuentran dentro del intervalo que se ingreso

Salida: Cuantos números primos había en dicho intervalo y los valores de los números primos

Restricciones: El intervalo debe ser entre números positivos

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Inicio del intervalo	Entero	N1
Final del intervalo	entero	N2
Contador	Entero	i

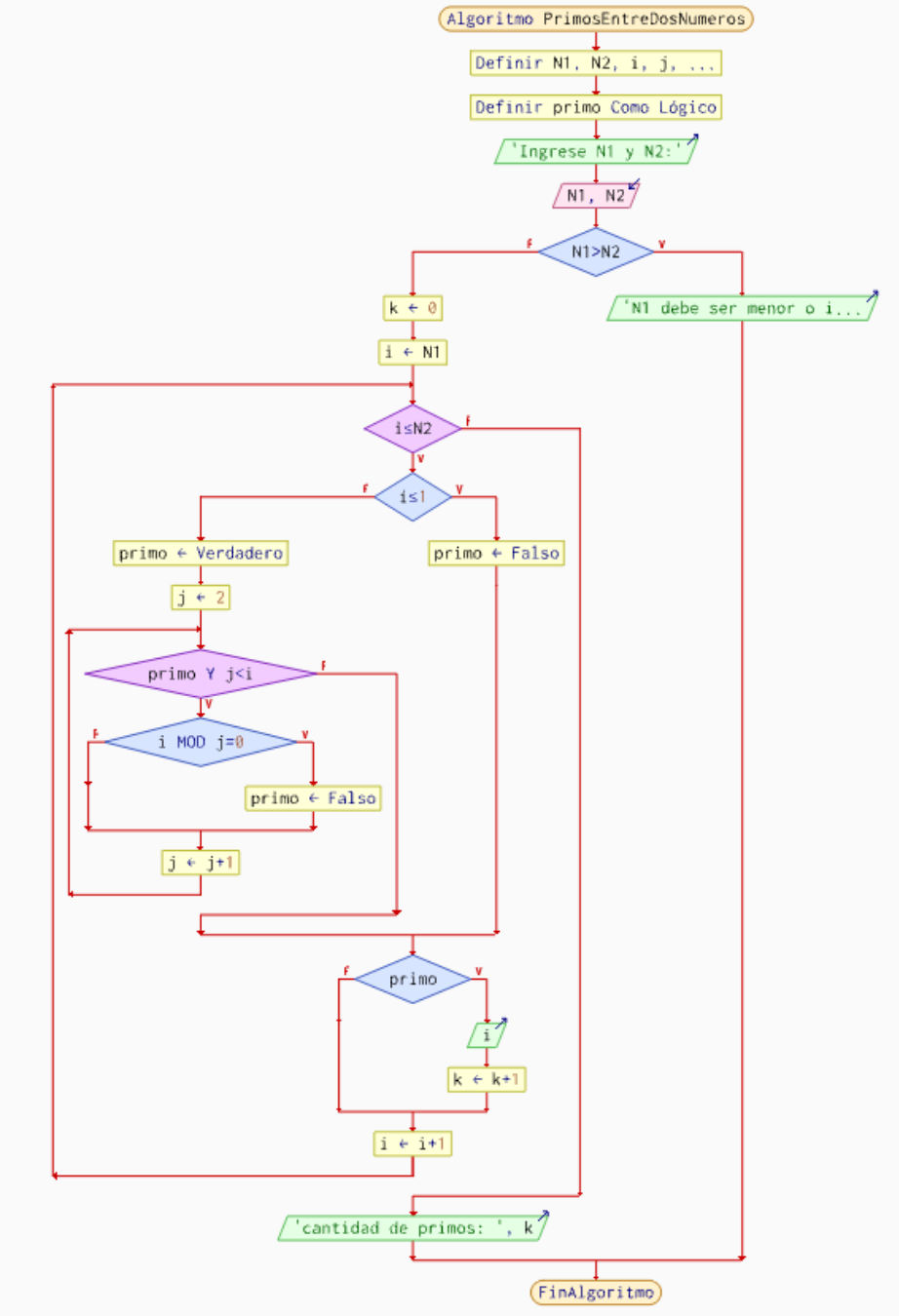
Número evaluado	Entero	j
Contador de divisores	Entero	k

Seudocódigo

```
Algoritmo PrimosEntreDosNumeros
        Definir N1, N2, i, j, k Como Entero;
        Definir primo Como Logico;
        Escribir "Ingrese N1 y N2:";
        Leer N1, N2;
        Si N1 > N2 Entonces
            Escribir "N1 debe ser menor o igual a N2";
        Sino
            k <- 0;
            i <- N1;
            Mientras i <= N2 Hacer
                Si i <= 1 Entonces
                    primo <- Falso;
                Sino
                    primo <- Verdadero;
                    j <- 2;
                    Mientras primo Y j < i Hacer
                        Si i % j = 0 Entonces
                            primo <- Falso;
                        FinSi
                        j <- j + 1;
                    FinMientras
                FinSi
                Si primo Entonces
                    Escribir i;
```

```
                k <- k + 1;
            FinSi
            i <- i + 1;
        FinMientras
        Escribir "cantidad de primos: ", k;
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	N1	N2	i	j	k	SALIDA
1	1	-	-	-	-	-
2	1	15	1	1	1	-
3	1	15	2	2	2	-
4	1	15	3	3	2	-
5	1	15	4	4	3	-
6	1	15	5	5	2	-

7	1	15	6	6	4	-
8	1	15	7	7	2	-
9	1	15	8	8	4	-
10	1	15	9	9	3	-
11	1	15	10	10	4	-
12	1	15	11	11	2	-
13	1	15	12	12	6	-
14	1	15	13	13	2	-
15	1	15	14	14	4	-
16	1	15	15	15	4	“Cantidad de primos : 13”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio9 Primo entre numeros

int main() {
    int N1, N2, i, j, k;
    int primo;

    printf("Ingrese N1 y N2: ");
    scanf("%d %d", &N1, &N2);

    if (N1 > N2) {
        printf("N1 debe ser menor o igual a N2\n");
    } else {
        k = 0;
        i = N1;

        while (i <= N2) {
            if (i <= 1) {
                primo = 0;
            } else {
                primo = 1;
                j = 2;
                while (primo && j < i) {
                    if (i % j == 0) {
                        primo = 0;
                    }
                    j = j + 1;
                }
            }

            if (primo) {
                printf("%d\n", i);
                k = k + 1;
            }

            i = i + 1;
        }
```

```
"C:\Users\sabes\OneDrive\De
Ingrese N1 y N2: 1 36
2
3
5
7
11
13
17
19
23
29
31
cantidad de primos: 11

Process returned 0 (0x0)   execution time : 4.138 s
Press any key to continue.
```

- Problema 1.10. Factorial

Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1 · 2 · … · n, sabiendo que 0! = 1.

Análisis del problema

Entrada: Un numero entero positivo mayor a 1

Proceso: Calcula la factorial del número ingresado

Salida: Se muestra el resultado de calcular la factorial del número ingresado

Restricciones: El numero ingresado debe ser entero, positivo y mayor a 1

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero ingresado	entero	n
Contador del ciclo	Entero	i
Resultado	Entero	resultado

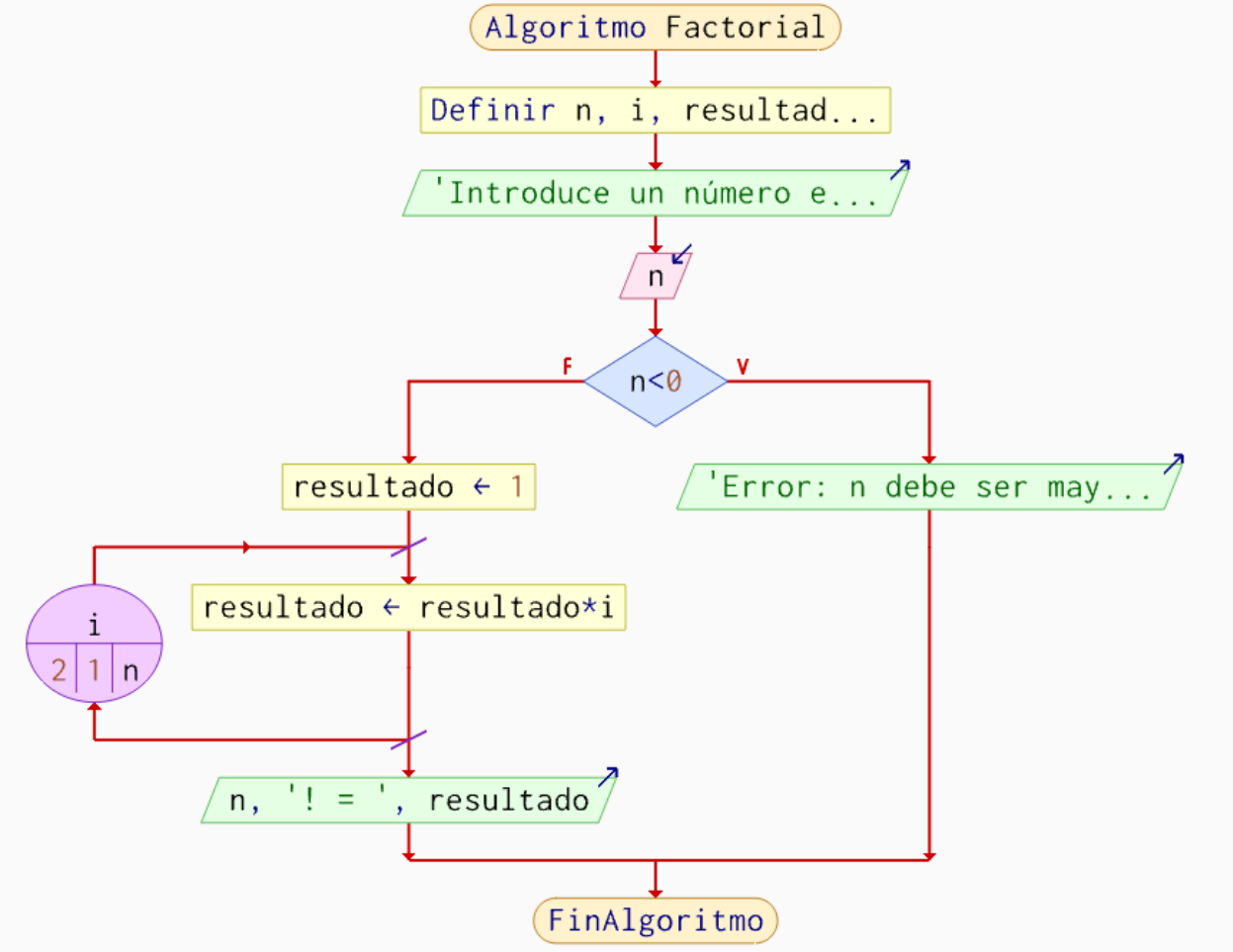
Seudocódigo

```
Algoritmo Factorial
        Definir n, i, resultado Como Entero;

        Escribir "Introduce un número entero n >= 0:";
        Leer n;

        Si n < 0 Entonces
            Escribir "Error: n debe ser mayor o igual que 0.";
        SiNo
            resultado <- 1;
            Para i <- 2 Hasta n Con Paso 1 Hacer
                resultado <- resultado * i;
            FinPara
            Escribir n, "! = ", resultado;
        FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	i	Resultado	SALIDA
1	4	-	-	-
2	4	1	1	-
3	4	2	1	-
4	4	2	1	‘
5	4	2	2	-
6	4	3	2	-
7	4	3	2	-
8	4	3	6	-
9	4	4	6	-
10	4	4	6	-
11	4	4	24	-
12	4	5	24	‘’4!= 24 ”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 10 factorial
int main(void) {
    int n;
    long long resultado = 1;

    printf("Introduce un numero entero n >= 0: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Error: entrada invalida o n < 0.\n");
        return 1;
    }

    for (int i = 2; i <= n; i++) {
        resultado *= i;
    }

    printf("%d! = %lld\n", n, resultado);

}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un numero entero n >= 0: 4
4! = 24

Process returned 0 (0x0)   execution time : 4.005 s
Press any key to continue.
```

- Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, …), donde cada término es la suma de los dos anteriores.

Análisis del problema

Entrada: Un numero entero positivo mayor a 1

Proceso: Calcula la serie de fibonacci

Salida: Muestra los primeros términos de la serie de fibonacci

Restricciones: El numero ingresado debe ser un entero positivo y mayor a 1

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numeros entero positivo mayor a 1	Entero	n
Primer termino actual	Entero	a
Segundo termino actual	Entero	b
Tercer termino calculado	Entero	c
Contador	Entero	i

Seudocódigo

Algoritmo Fibonacci

Definir n, i, a, b, c Como Entero;

Escribir "Introduce un entero n >= 1:";

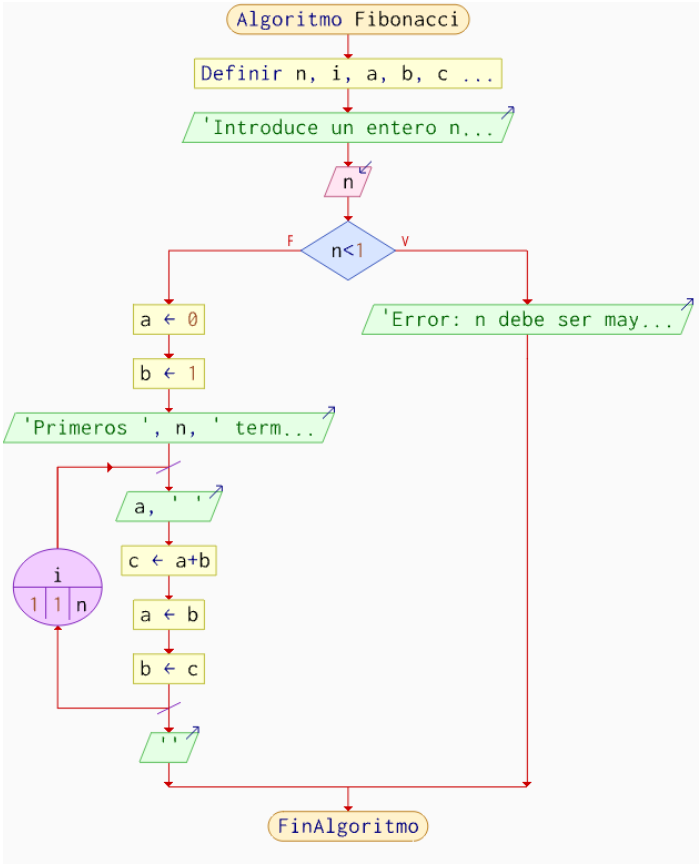
Leer n;

```
        Si n < 1 Entonces
            Escribir "Error: n debe ser mayor o igual que 1.";
        SiNo
            a <- 0;
            b <- 1;

            Escribir "Primeros ", n, " terminos de Fibonacci:";

            Para i <- 1 Hasta n Con Paso 1 Hacer
                Escribir Sin Saltar a, " ";
                c <- a + b;
                a <- b;
                b <- c;
            FinPara
            Escribir "";
        FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	i	a	b	c	SALIDA
1	6	-	-	-	-	-
2	6	-	0	1	-	-
3	6	3	0	1	1	-
4	6	3	0	1	1	-
5	6	3	1	1	1	-
6	6	4	1	1	1	-
7	6	4	1	1	2	-
8	6	4	1	2	2	-
9	6	5	1	2	2	-
10	6	5	1	2	3	-
11	6	5	2	2	3	-
12	6	5	2	3	3	-
13	6	6	2	3	3	-
14	6	6	2	3	5	-
15	6	6	3	3	5	-
16	6	6	3	5	5	-
17	6	7	3	5	5	“0,1,1,2,3,5”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio 11
int main(void) {
    int n;

    printf("Introduce un entero n >= 1: ");
    if (scanf("%d", &n) != 1 || n < 1) {
        printf("Error: entrada invalida o n < 1.\n");
        return 1;
    }

    long long a = 0, b = 1, c;

    printf("Primeros %d terminos de Fibonacci:\n", n);
    for (int i = 1; i <= n; i++) {
        printf("%lld", a);
        if (i < n) printf(", ");
        c = a + b;
        a = b;
        b = c;
    }
    printf("\n");
    return 0;
}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero n >= 1: 5
Primeros 5 terminos de Fibonacci:
0, 1, 1, 2, 3

Process returned 0 (0x0)   execution time : 3.376 s
Press any key to continue.
```

- Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.

Análisis del problema

Entrada: Un numero entero positivo

Proceso: Calcula la suma de sus dígitos

Salida: Muestra el resultado de la suma de los dígitos del numero ingresado

Restricciones: el numero ingresado debe ser un entero positivo

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Número ingresado	Entero	n
Suma de dígitos	Entero	suma
Ultimo digito	Entero	digito

Seudocódigo

Algoritmo SumaDigitos

Definir n, suma, digito Como Entero;

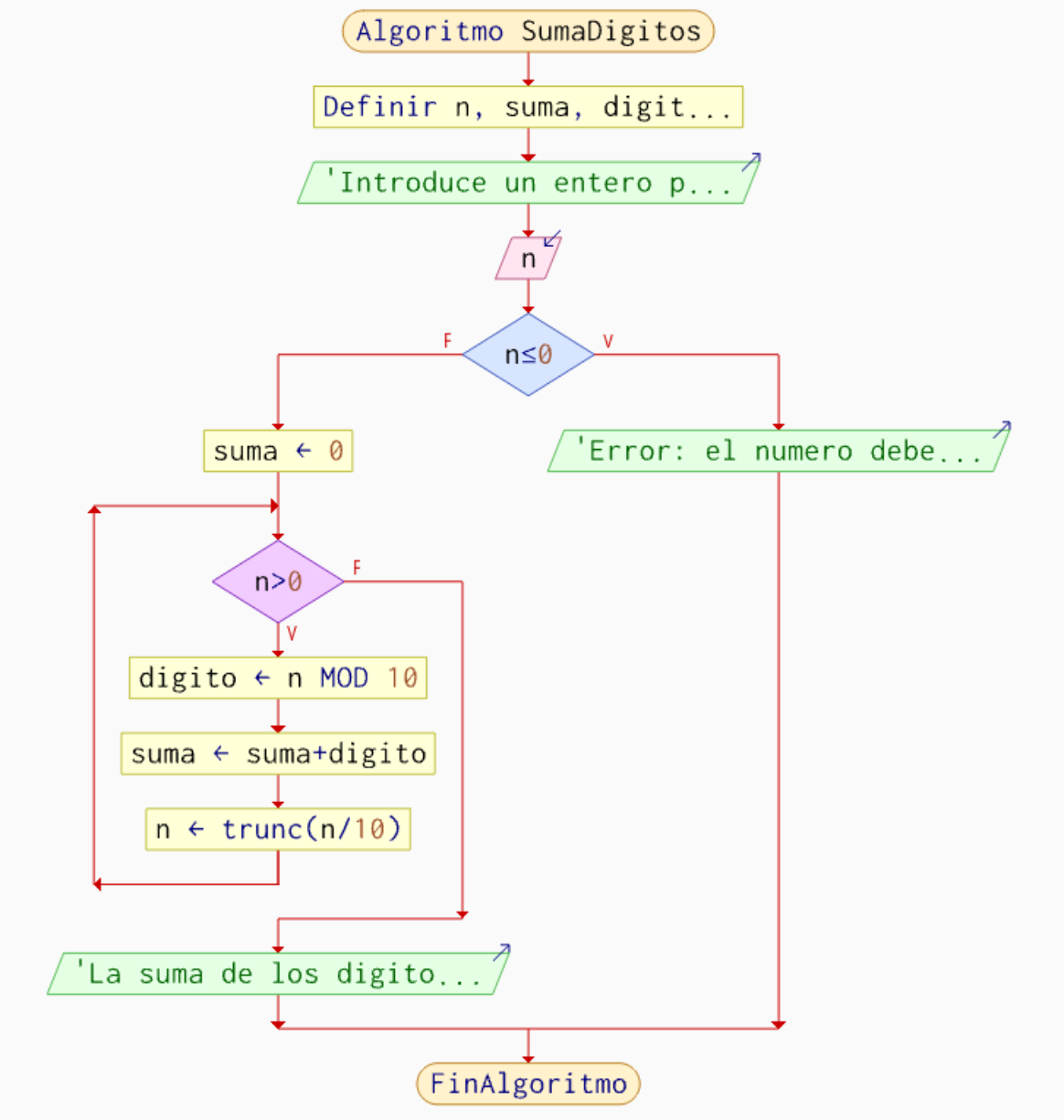
Escribir "Introduce un entero positivo:";

```
        Leer n;

        Si n <= 0 Entonces
            Escribir "Error: el numero debe ser positivo.";
        SiNo
            suma <- 0;
            Mientras n > 0 Hacer
                digito <- n MOD 10;
                suma <- suma + digito;
                n <- trunc(n / 10);
            FinMientras

            Escribir "La suma de los digitos es: ", suma;
        FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	digito	suma	SALIDA
1	678	-	-	-
2	678	-	0	-
3	678	8	0	-
4	678	8	8	-
5	67	7	8	-
6	67	7	15	-
7	6	6	15	-
8	6	6	21	-
9	0	6	21	“La suma de los dígitos es : 21”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio12 sumadigitos
int main(void) {
    long long n;
    int suma = 0;

    printf("Introduce un entero positivo: ");
    if (scanf("%lld", &n) != 1 || n <= 0) {
        printf("Error: entrada invalida o numero no positivo.\n");
        return 1;
    }

    long long original = n;

    while (n > 0) {
        suma += n % 10;
        n /= 10;
    }

    printf("La suma de los digitos de %lld es: %d\n", original, suma);
    return 0;
}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero positivo: 678
La suma de los digitos de 678 es: 21

Process returned 0 (0x0)   execution time : 3.672
Press any key to continue.
```

- Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

Análisis del problema

Entrada: Un numero entero positivo mayor a 0

Proceso: Calcular si el numero ingresado se lee igual de izquierda a derecha y de derecha a izquierda

Salida: Mostrar si el numero es capicúa o no

Restricciones: el numero ingresado debe ser un entero positivo mayor a 0

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero ingresado	Entero	n
Numero original	Entero	original
Numero al revez	Entero	invertido
Ultimo digito	Entero	digito

Seudocódigo

Algoritmo Capicua

Definir n, original, invertido, digito Como Entero;

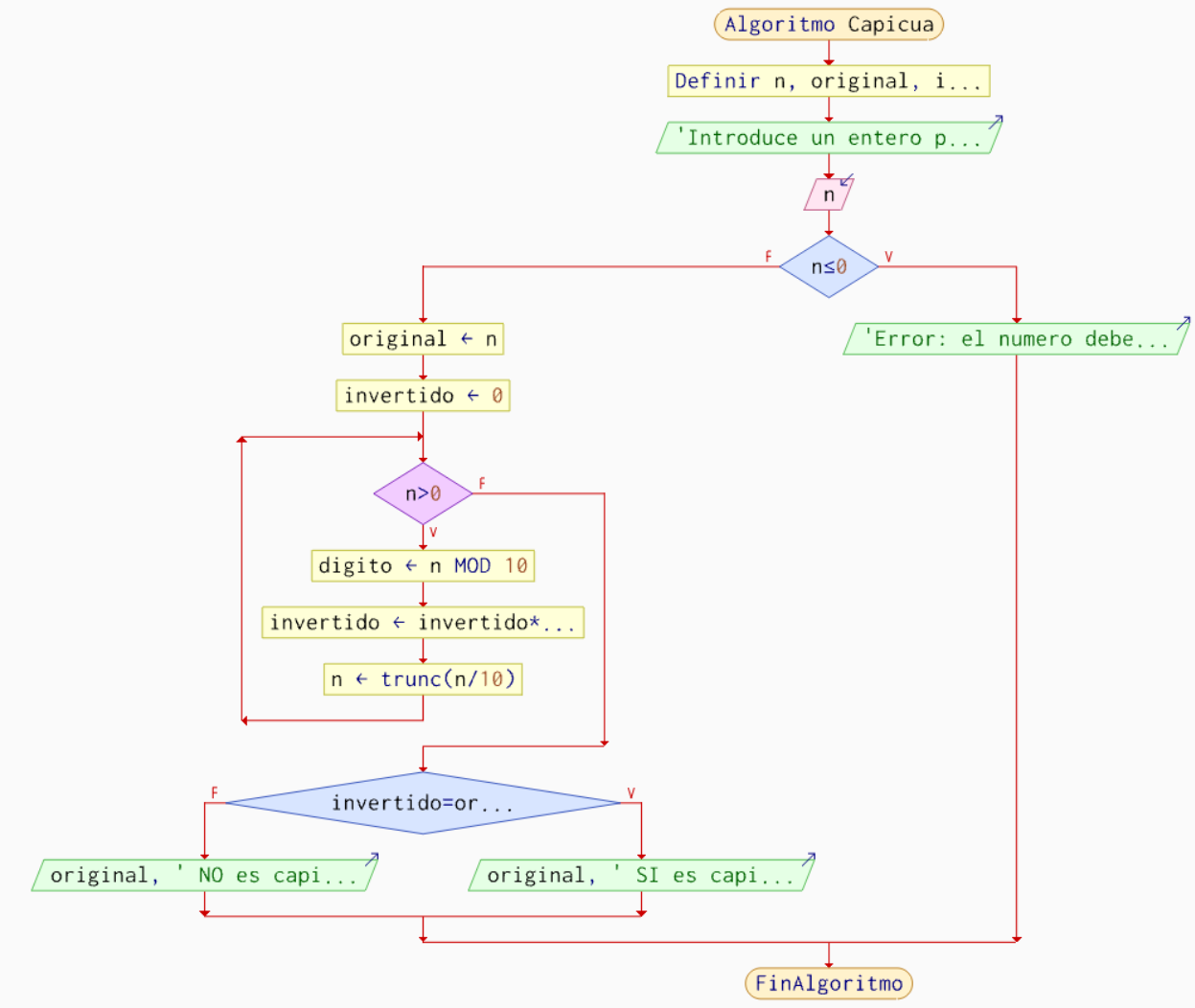
```
    Escribir "Introduce un entero positivo:";
    Leer n;

    Si n <= 0 Entonces
        Escribir "Error: el numero debe ser positivo.";
    SiNo
        original <- n;
        invertido <- 0;

        Mientras n > 0 Hacer
            digito <- n MOD 10;
            invertido <- invertido * 10 + digito;
            n <- trunc(n / 10);
        FinMientras

        Si invertido = original Entonces
            Escribir original, " SI es capicua.";
        SiNo
            Escribir original, " NO es capicua.";
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	digito	invertido	SALIDA
1	1221	-	-	-
2	1221	-	0	-
3	1221	1	0	-
4	1221	1	1	-
5	122	1	1	-
6	122	2	1	-
7	122	2	12	-
8	12	2	12	-
9	12	2	122	-
10	1	2	122	-
11	1	1	122	-
12	1	1	1221	-
13	0	1	1221	“1221 Si es capicúa”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio13 capicua
int main(void) {
    long long n, original, invertido = 0;
    int digito;

    printf("Introduce un entero positivo: ");
    if (scanf("%lld", &n) != 1 || n <= 0) {
        printf("Error: entrada invalida o numero no positivo.\n");
        return 1;
    }

    original = n;

    while (n > 0) {
        digito = n % 10;
        invertido = invertido * 10 + digito;
        n /= 10;
    }

    if (invertido == original)
        printf("%lld SI es capicua.\n", original);
    else
        printf("%lld NO es capicua.\n", original);

    return 0;
}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero positivo: 665443
665443 NO es capicua.

Process returned 0 (0x0)   execution time : 7.257 s
Press any key to continue.
```

- Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD.

Análisis del problema

Entrada: Ingresar dos números positivos a y b donde b tiene que ser distinto a 0

Proceso: Calcula el máximo común divisor con el método del Euclides

Salida: El valor del máximo común divisor de los dos números ingresados

Restricciones: Los números que se ingresan deben ser números enteros positivos

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
1er Numero ingresado	Entero	a
2do Numero ingresado	Entero	b
Residuo	Entero	temp

Seudocódigo

Algoritmo max_comun_divisor

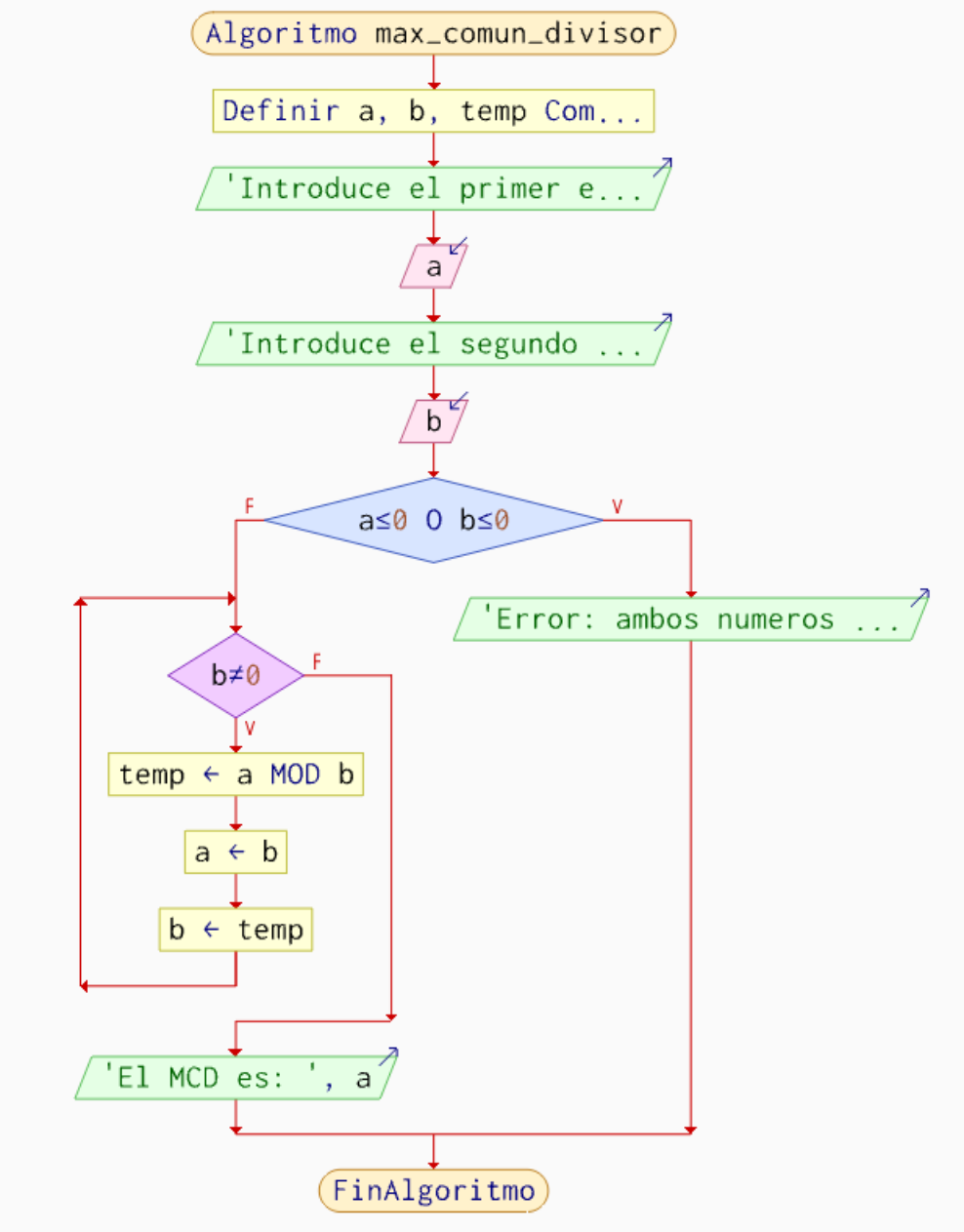
Definir a, b, temp Como Entero;

```
        Escribir "Introduce el primer entero positivo:";
        Leer a;
        Escribir "Introduce el segundo entero positivo:";
        Leer b;

        Si a <= 0 O b <= 0 Entonces
            Escribir "Error: ambos numeros deben ser positivos.";
        SiNo
            Mientras b <> 0 Hacer
                temp <- a MOD b;
                a <- b;
                b <- temp;
            FinMientras

            Escribir "El MCD es: ", a;
        FinSi
FinAlgoritmo
```

Diagrama de flujo

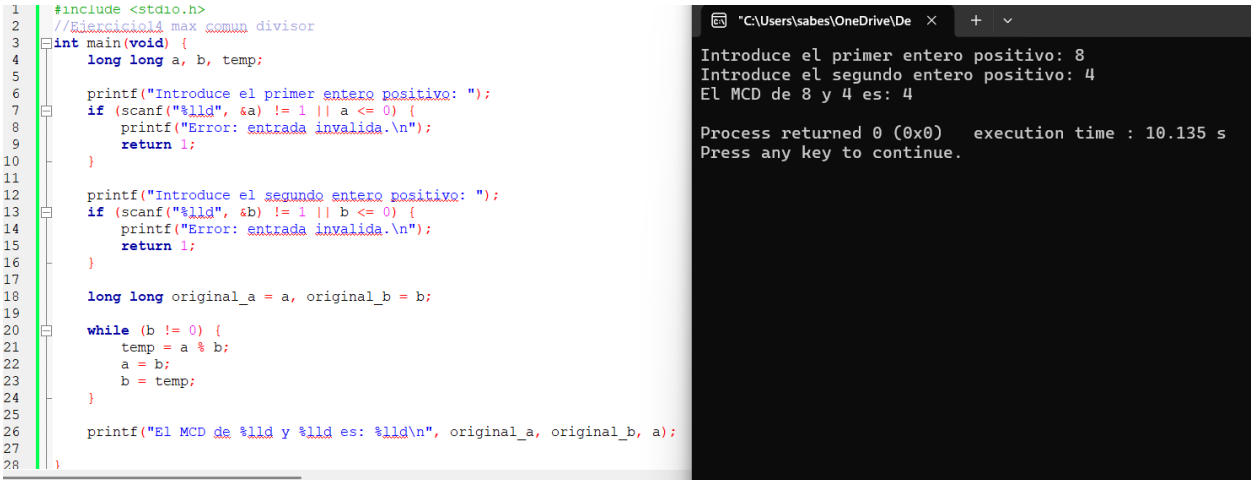


Prueba de escritorio

PASO	a	b	temp	SALIDA
1	48	18	-	-
2	48	18	12	-
3	18	18	12	-
4	18	12	12	-
5	18	12	6	-
6	12	12	6	-
7	12	6	6	-
8	12	6	0	-

9	6	6	0	-
10	6	0	0	“El MCD es 6”

Ejecutado Codeblocks



```c
#include <stdio.h>
//Ejercicio14 max comun divisor
int main(void) {
    long long a, b, temp;

    printf("Introduce el primer entero positivo: ");
    if (scanf("%lld", &a) != 1 || a <= 0) {
        printf("Error: entrada invalida.\n");
        return 1;
    }

    printf("Introduce el segundo entero positivo: ");
    if (scanf("%lld", &b) != 1 || b <= 0) {
        printf("Error: entrada invalida.\n");
        return 1;
    }

    long long original_a = a, original_b = b;

    while (b != 0) {
        temp = a % b;
        a = b;
        b = temp;
    }

    printf("El MCD de %lld y %lld es: %lld\n", original_a, original_b, a);

}
```

```
"C:\Users\sabes\OneDrive\De
Introduce el primer entero positivo: 8
Introduce el segundo entero positivo: 4
El MCD de 8 y 4 es: 4

Process returned 0 (0x0)   execution time : 10.135 s
Press any key to continue.
```

- Problema 1.15. Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible

entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

Análisis del problema

Entrada: Un numero entero que represente un año que querremos comprobar

Proceso: Comprobar si el año ingresado es un ano bisiesto o no

Salida: Mostrar si año ingresado es bisiesto o no

Restricciones: Debe ser un numero entero positivo

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Año para analizar	Entero	anio

Seudocódigo

Algoritmo Bisiesto

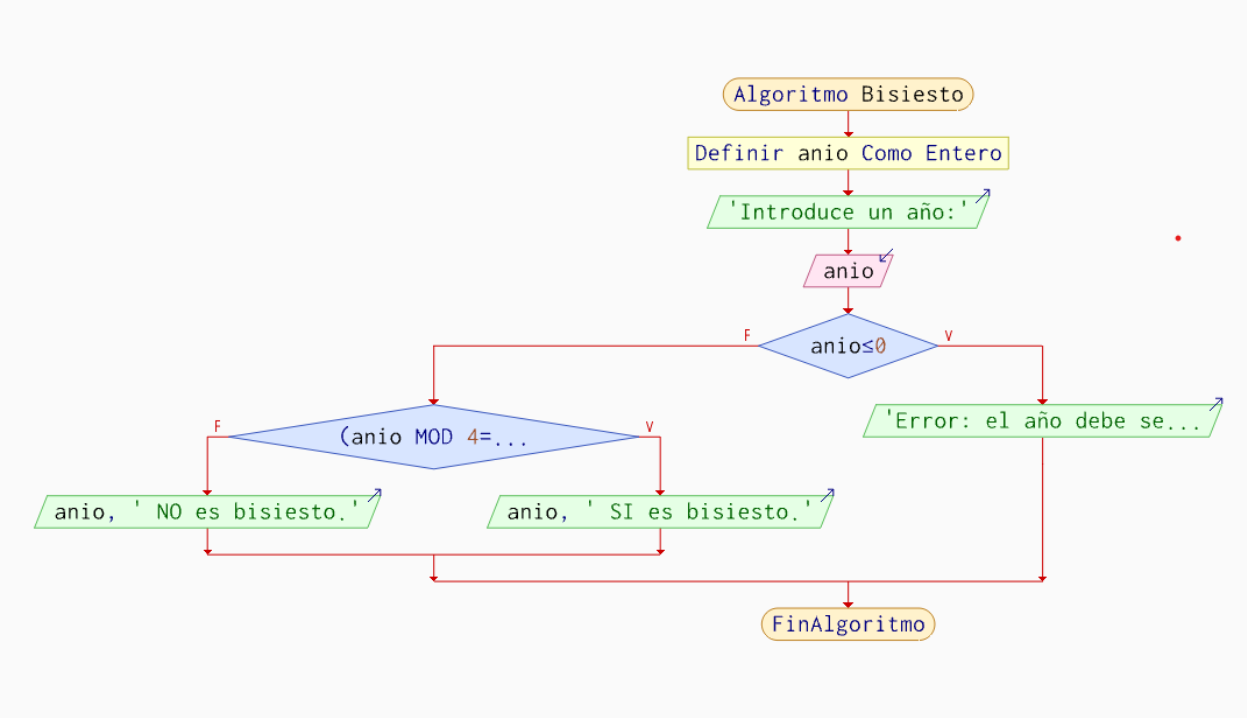
Definir anio Como Entero;

```
        Escribir "Introduce un año:";
        Leer anio;

        Si anio <= 0 Entonces
            Escribir "Error: el año debe ser un entero positivo.";
        SiNo
            Si (anio MOD 4 = 0 Y anio MOD 100 <> 0) O (anio MOD 400 = 0)
Entonces
                Escribir anio, " SI es bisiesto.";
            SiNo
                Escribir anio, " NO es bisiesto.";
            FinSi
        FinSi

FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

CASO	año	Divisible/4	Divisible/100	Divisible/400	SALIDA
1	2024	Verdadero	falso	-	“Bisiesto”
2	1900	Verdadero	Verdadero	Falso	“No es bisiesto”
3	2000	Verdadero	Verdadero	Verdadero	“Bisiesto”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio15
int main(void) {
    int anio;

    printf("Introduce un anio: ");
    if (scanf("%d", &anio) != 1 || anio <= 0) {
        printf("Error: entrada invalida o anio no positivo.\n");
        return 1;
    }

    if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
        printf("%d SI es bisiesto.\n", anio);
    } else {
        printf("%d NO es bisiesto.\n", anio);
    }

    return 0;
}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un anio: 2000
2000 SI es bisiesto.

Process returned 0 (0x0)   execution time : 5.336 s
Press any key to continue.
```

- Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

Análisis del problema

Entrada: Un numero entero positivo mayor a 0

Proceso: Determinar si el numero es igual a la suma de sus divisores propios

Salida: Mostrar si el numero es perfecto o no

Restricciones: El numero ingresado debe ser positivo y mayor a 0

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero ingresado	Entero	n
Suma de divisores	Entero	suma
contador	Entero	i

Seudocódigo

```
Algoritmo NumeroPerfecto
        Definir n, suma, i Como Entero;
        Escribir "Introduce un entero positivo:";
        Leer n;

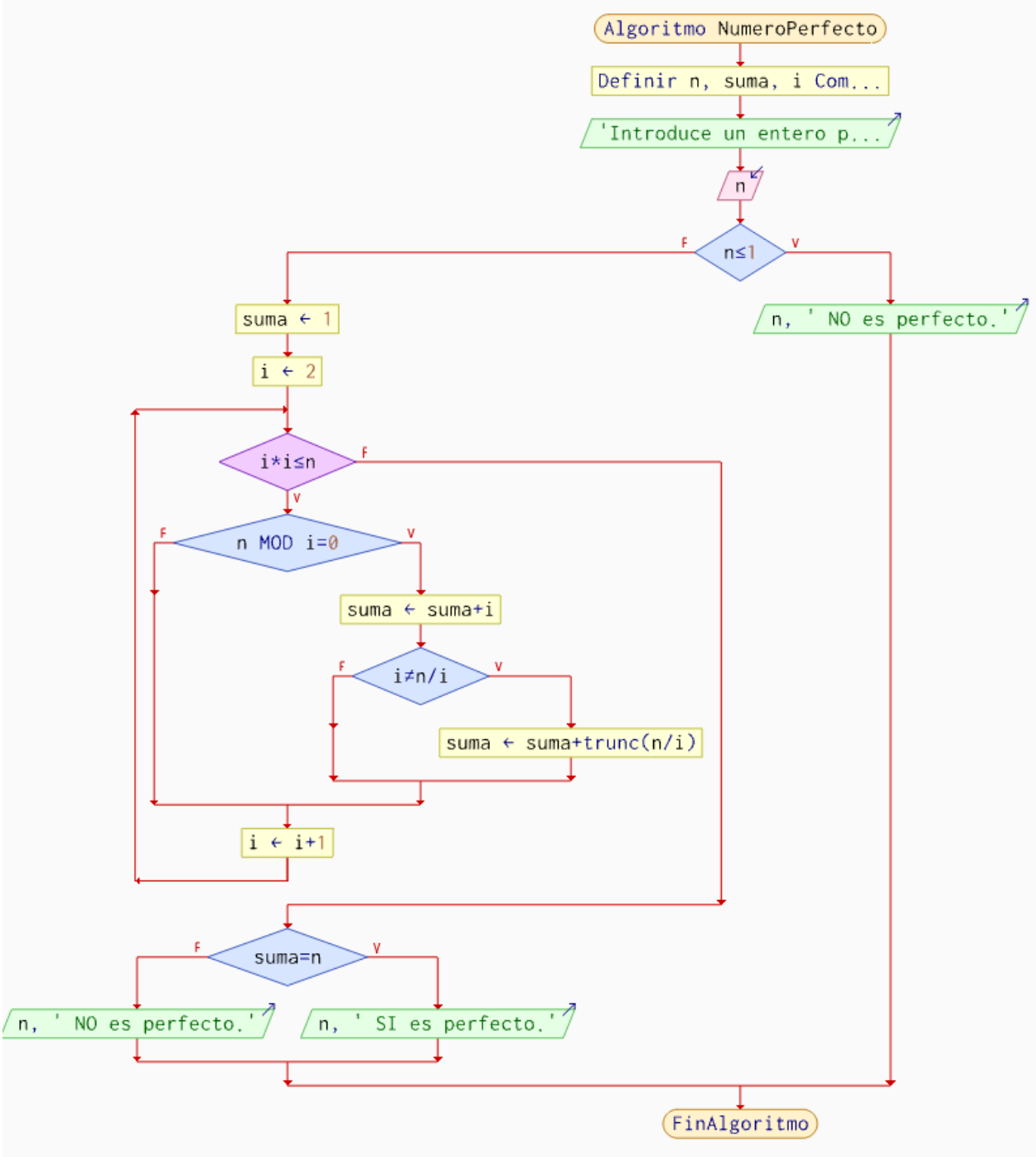
        Si n <= 1 Entonces
            Escribir n, " NO es perfecto.";
        SiNo
            suma <- 1;
            i <- 2;

            Mientras i * i <= n Hacer
                Si n MOD i = 0 Entonces
                    suma <- suma + i;
                    Si i <> n / i Entonces
                        suma <- suma + trunc(n / i);
                    FinSi
                FinSi
```

```
            i <- i + 1;
        FinMientras

        Si suma = n Entonces
            Escribir n, " SI es perfecto.";
        SiNo
            Escribir n, " NO es perfecto.";
        FinSi
    FinSi
FinAlgoritmo
```

Diagrama de flujo

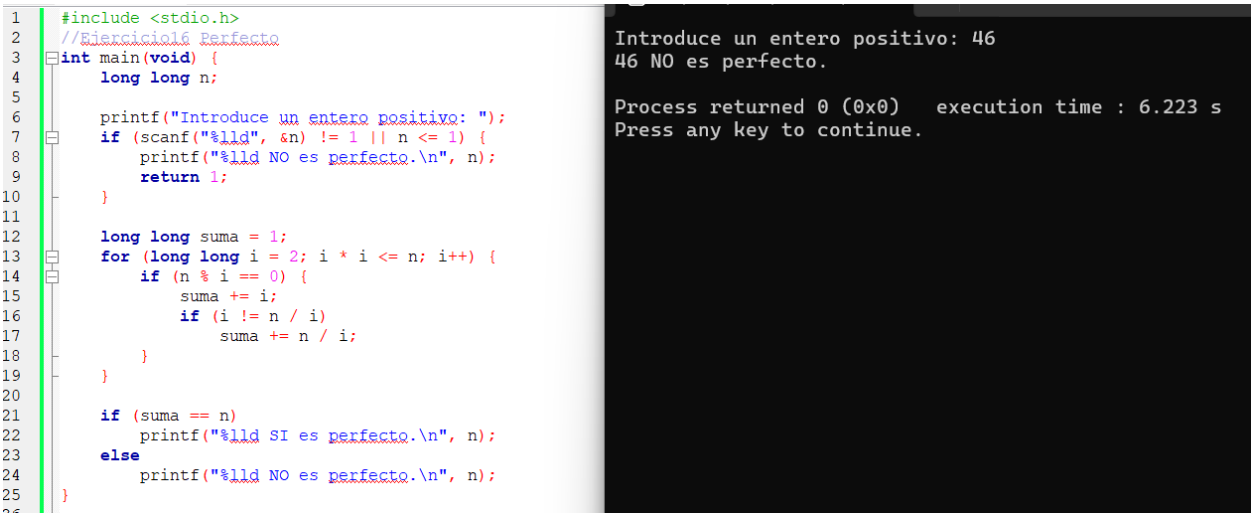


Prueba de escritorio

PASO	num	i	sum	SALIDA
1	6	-	-	-
2	6	1	0	-
3	6	1	1	-
4	6	2	1	-
5	6	2	3	-

6	6	3	3	-
7	6	3	6	-
8	6	4	6	-
9	6	5	6	-
10	6	6	6	“6 Si es un número Perfecto”

Ejecutado Codeblocks



```
#include <stdio.h>
//Ejercicio16 Perfecto
int main(void) {
    long long n;

    printf("Introduce un entero positivo: ");
    if (scanf("%lld", &n) != 1 || n <= 1) {
        printf("%lld NO es perfecto.\n", n);
        return 1;
    }

    long long suma = 1;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            suma += i;
            if (i != n / i)
                suma += n / i;
        }
    }

    if (suma == n)
        printf("%lld SI es perfecto.\n", n);
    else
        printf("%lld NO es perfecto.\n", n);
}
```

```
Introduce un entero positivo: 46
46 NO es perfecto.

Process returned 0 (0x0)   execution time : 6.223 s
Press any key to continue.
```

- Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

Análisis del problema

Entrada: Un número entero positivo en base 10

Proceso: Calcular el numero entero a su representación en binario

Salida: Muestra la representación del número ingresado a binario

Restricciones: El numero ingresado debe ser positivo y estar en base 10

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Número ingresado	Entero	n
Resto	Entero	resto
Resultado binario	Cadena	binario

Seudocódigo

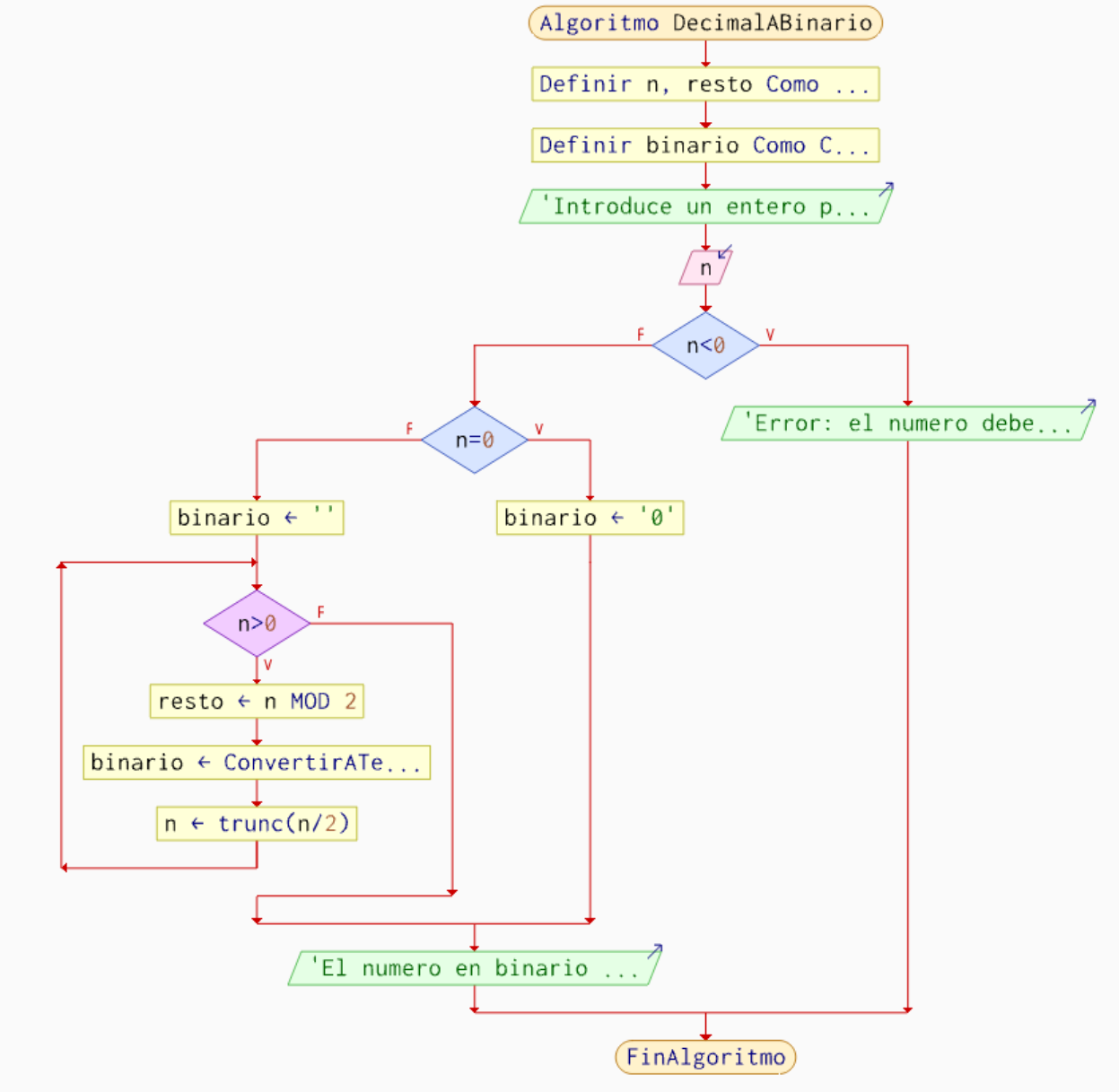
```
Algoritmo DecimalABinario
        Definir n, resto Como Entero;
        Definir binario Como Cadena;

        Escribir "Introduce un entero positivo:";
        Leer n;
        Si n < 0 Entonces
            Escribir "Error: el numero debe ser positivo o cero.";
        SiNo
            Si n = 0 Entonces
                binario <- "0";
            SiNo
                binario <- "";
                Mientras n > 0 Hacer
                    resto <- n MOD 2;
                    binario <- ConvertirATexto(resto) + binario;
                    n <- trunc(n / 2);
                FinMientras
            FinSi

            Escribir "El numero en binario es: ", binario;
        FinSi

FinAlgoritmo
```

Diagrama de flujo

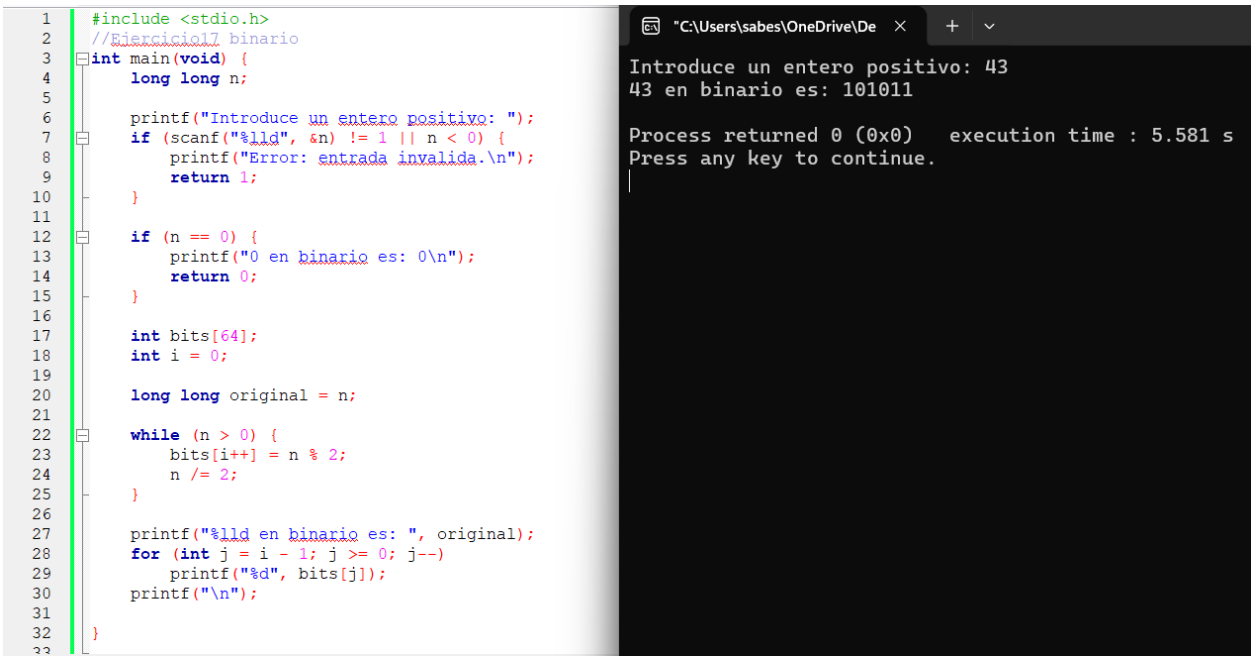


Prueba de escritorio

PASO	n	resto	binario	SALIDA
1	13	-	-	-
2	13	1	-	-
3	13	1	1	-
4	6	1	1	-
5	6	0	1	-
6	6	0	01	-
7	3	0	01	-
8	3	1	01	-
9	3	1	101	-
10	1	1	101	-

11	1	1	1101	-
12	0	1	1101	“En binario es : 1101”

Ejecutado Codeblocks



```c
#include <stdio.h>
//Ejercicio17 binario
int main(void) {
    long long n;

    printf("Introduce un entero positivo: ");
    if (scanf("%lld", &n) != 1 || n < 0) {
        printf("Error: entrada invalida.\n");
        return 1;
    }

    if (n == 0) {
        printf("0 en binario es: 0\n");
        return 0;
    }

    int bits[64];
    int i = 0;

    long long original = n;

    while (n > 0) {
        bits[i++] = n % 2;
        n /= 2;
    }

    printf("%lld en binario es: ", original);
    for (int j = i - 1; j >= 0; j--)
        printf("%d", bits[j]);
    printf("\n");

}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero positivo: 43
43 en binario es: 101011

Process returned 0 (0x0)   execution time : 5.581 s
Press any key to continue.
```

- Problema 1.18. Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.

Análisis del problema

Entrada: Un numero entero mayor o igual a 1

Proceso: Dibujar un rectángulo de numero de filas igual al número ingresado

Salida: Mostrar rectángulo de asteriscos de filas igual al número ingresado

Restricciones: El numero ingresado debe ser un numero entero mayor a 1

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Numero de filas	Entero	n
Contador de filas	Entero	i

Contador de asteriscos	Entero	j

Seudocódigo

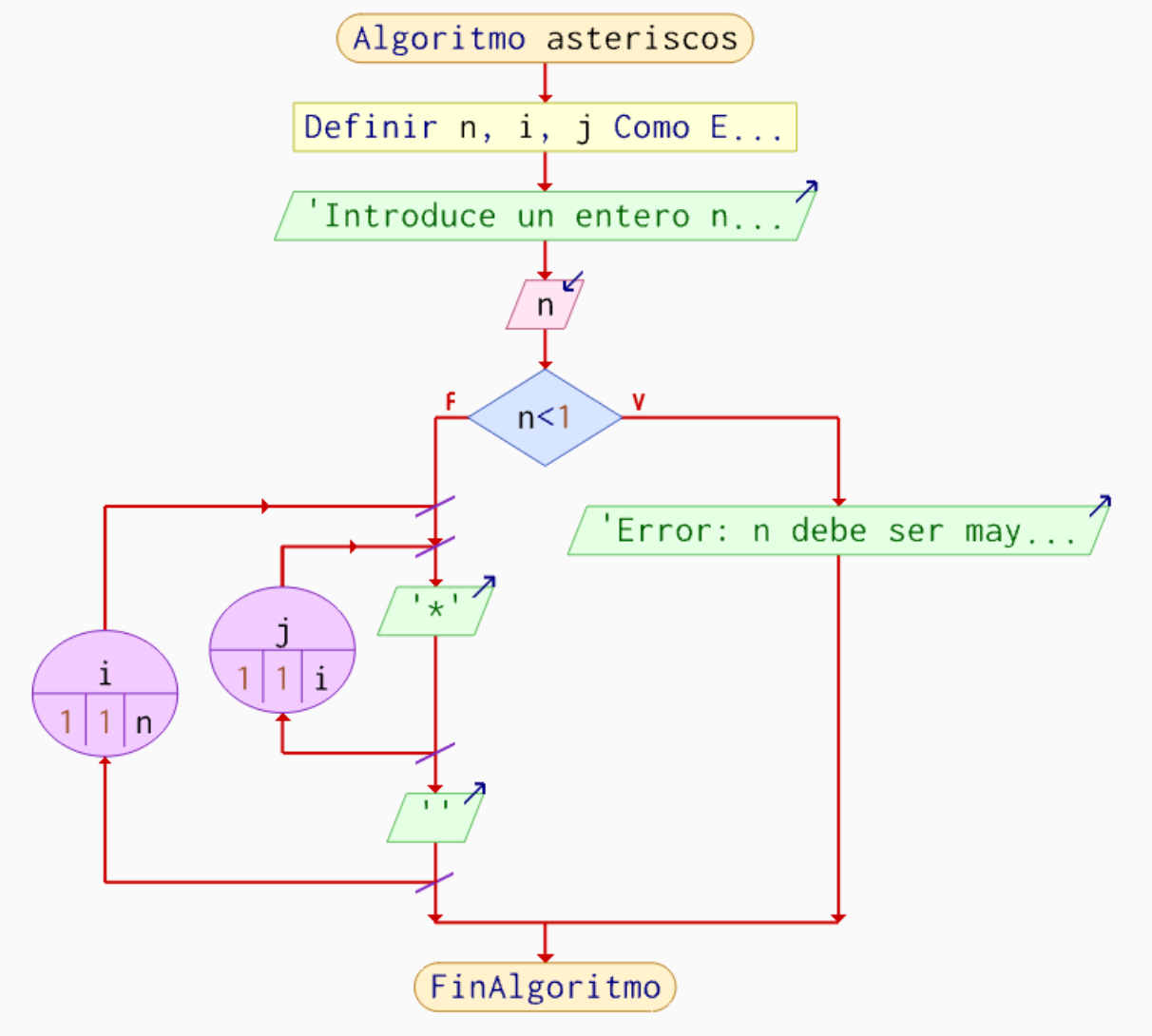
```
Algoritmo asteriscos
        Definir n, i, j Como Entero;

        Escribir "Introduce un entero n >= 1:";
        Leer n;

        Si n < 1 Entonces
            Escribir "Error: n debe ser mayor o igual que 1.";
        SiNo
            Para i <- 1 Hasta n Con Paso 1 Hacer
                Para j <- 1 Hasta i Con Paso 1 Hacer
                    Escribir Sin Saltar "*";
                FinPara
                Escribir "";
            FinPara
        FinSi

FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	i	j	SALIDA
1	4	1	-	-
2	4	1	1	*
3	4	1	2	-
4	4	2	2	-
5	4	2	1	-
6	4	2	1	*
7	4	2	2	-
8	4	2	2	**
9	4	2	3	-
10	4	3	3	-
11	4	3	1	-
12	4	3	1	*
13	4	3	2	-

14	4	3	3	***
15	4	3	4	-
16	4	4	4	-
17	4	4	1	-
18	4	4	2	-
19	4	4	3	***
20	4	4	4	-
21	4	4	4	****

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio19 asteriscos
int main(void) {
    int n;

    printf("Introduce un entero n >= 1: ");
    if (scanf("%d", &n) != 1 || n < 1) {
        printf("Error: entrada invalida o n < 1.\n");
        return 1;
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            putchar('*');
        }
        putchar('\n');
    }
}
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero n >= 1: 4
*
**
***
****

Process returned 0 (0x0)   execution time : 4.670 s
Press any key to continue.
```

- Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

Análisis del problema

Entrada: Números enteros positivos hasta que se introduzca un 0 para finalizar el programa

Proceso: Calcula cual es el número y el segundo mayores de los valores ingresados

Salida: Mostrar cual es el mayor y el segundo mayor número de los ingresados

Restricciones: Los números ingresados deben ser positivos

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Números ingresados	Entero	n
Valor mayor	Entero	mayor

Segundo mayor	Entero	segundo
Contador	Entero	cantidad

Seudocódigo

```
Algoritmo MayorYSegundoMayor
    Definir n, mayor, segundo, cantidad Como Entero;
    Definir hayMayor, haySegundo Como Logico;
    hayMayor <- Falso;
    haySegundo <- Falso;
    cantidad <- 0;
    Repetir
        Escribir "Introduce un entero positivo (0 para terminar):";
        Leer n;

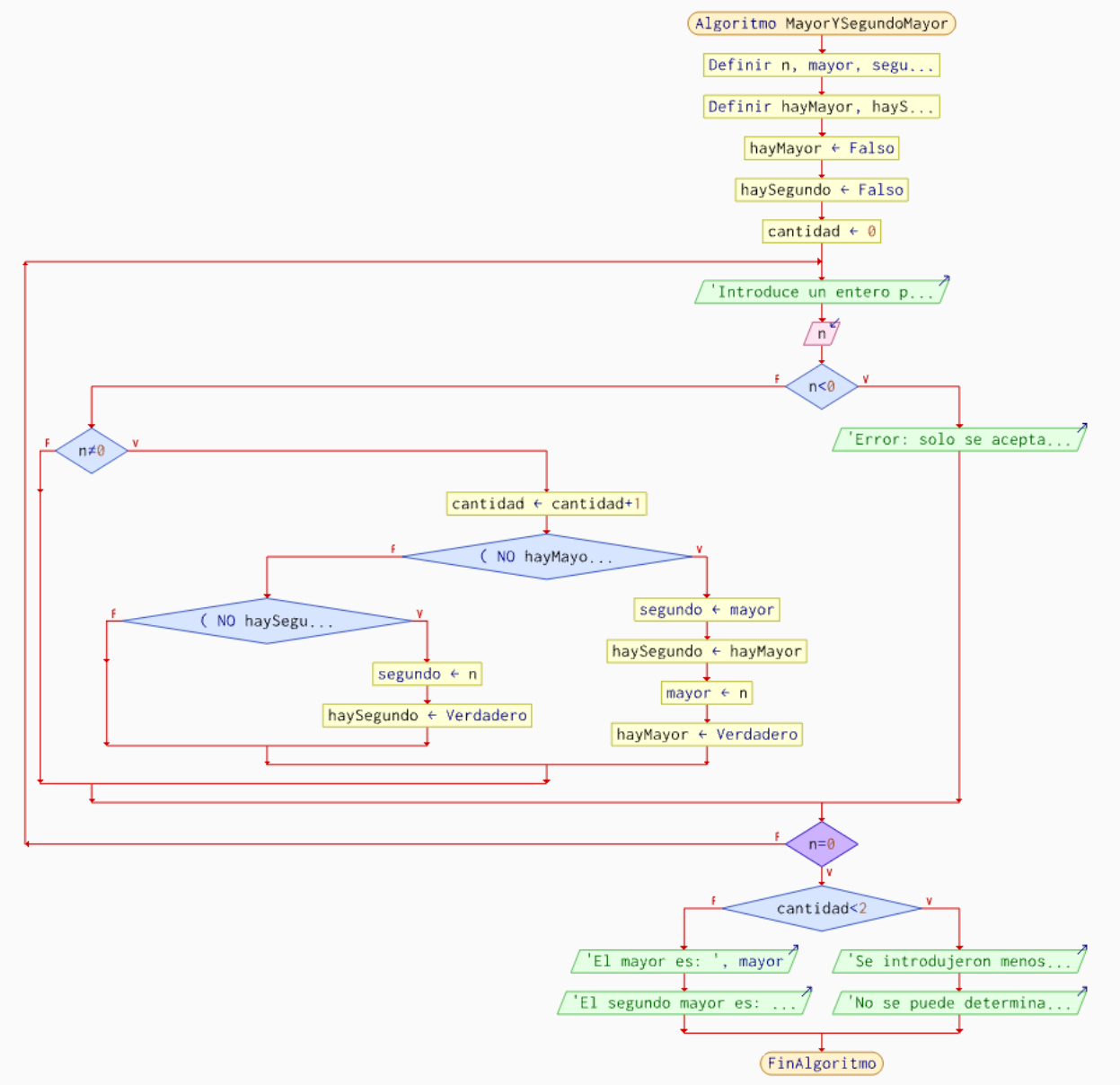
        Si n < 0 Entonces
            Escribir "Error: solo se aceptan enteros positivos.";
        SiNo
            Si n <> 0 Entonces
                cantidad <- cantidad + 1;

                Si (NO hayMayor) O (n > mayor) Entonces
                    segundo <- mayor;
                    haySegundo <- hayMayor;
                    mayor <- n;
                    hayMayor <- Verdadero;
                SiNo
                    Si (NO haySegundo) O (n > segundo) Entonces
                        segundo <- n;
                        haySegundo <- Verdadero;
```

```
                    FinSi
                FinSi
            FinSi
        FinSi
    Hasta Que n = 0

    Si cantidad < 2 Entonces
        Escribir "Se introdujeron menos de dos valores.";
        Escribir "No se puede determinar el segundo mayor.";
    SiNo
        Escribir "El mayor es: ", mayor;
        Escribir "El segundo mayor es: ", segundo;
    FinSi
FinAlgoritmo
```

Diagrama de flujo



Prueba de escritorio

PASO	n	cantidad	mayor	segundo	SALIDA
1	15	0	0	0	-
2	15	1	15	0	-
3	8	1	15	0	-
4	8	2	15	8	-
5	22	2	15	8	-
6	22	3	22	15	-
7	5	3	22	15	-
8	5	4	22	15	-
9	0	4	22	15	“El mayor es 22” “El segundo mayor es 15”

Ejecutado Codeblocks

```c
#include <stdio.h>
//Ejercicio19 mayor y seg mayor
int main(void) {
    long long n;
    long long mayor = 0, segundo = 0;
    int hayMayor = 0, haySegundo = 0;
    int cantidad = 0;

    while (1) {
        printf("Introduce un entero positivo (0 para terminar): ");
        if (scanf("%lld", &n) != 1) {
            printf("Error: entrada invalida.\n");
            while (getchar() != '\n');
            continue;
        }

        if (n == 0) break;

        if (n < 0) {
            printf("Error: solo se aceptan enteros positivos.\n");
            continue;
        }

        cantidad++;

        if (!hayMayor || n > mayor) {
            segundo = mayor;
            haySegundo = hayMayor;
            mayor = n;
            hayMayor = 1;
        } else if (!haySegundo || n > segundo) {
            segundo = n;
```

```
"C:\Users\sabes\OneDrive\De
Introduce un entero positivo (0 para terminar): 6
Introduce un entero positivo (0 para terminar): 3
Introduce un entero positivo (0 para terminar): 18
Introduce un entero positivo (0 para terminar): 89
Introduce un entero positivo (0 para terminar): 0
El mayor es: 89
El segundo mayor es: 18

Process returned 0 (0x0)   execution time : 19.130 s
Press any key to continue.
```

- Problema 1.20. Adivina el número

Desarrolla un programa que “piense” un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar “más alto”, “más bajo” o “¡acertaste!”, y al final mostrar cuántos intentos se usaron.

Análisis del problema

Entrada: Números de intentos del usuario

Proceso: Comparar los números ingresados por el usuario con el numero escogido

Salida: Mostar si el numero ingresado es mayor menor o igual al número secreto

Restricciones: El numero ingresado debe estar entre 1 a 100

Tabla de objetos

OBJETO	*TIPO*	*NOMBRE*
Número secreto	Entero	objetivo
Número ingresado	Entero	intento
Contador de intentos	Entero	intentos

Seudocódigo

```
Algoritmo AdivinarNumero
        Definir objetivo, intento, intentos Como Entero;
        objetivo <- 42;
        intentos <- 0;

        Escribir "He pensado un numero entre 1 y 100. ¡Adivinalo!";

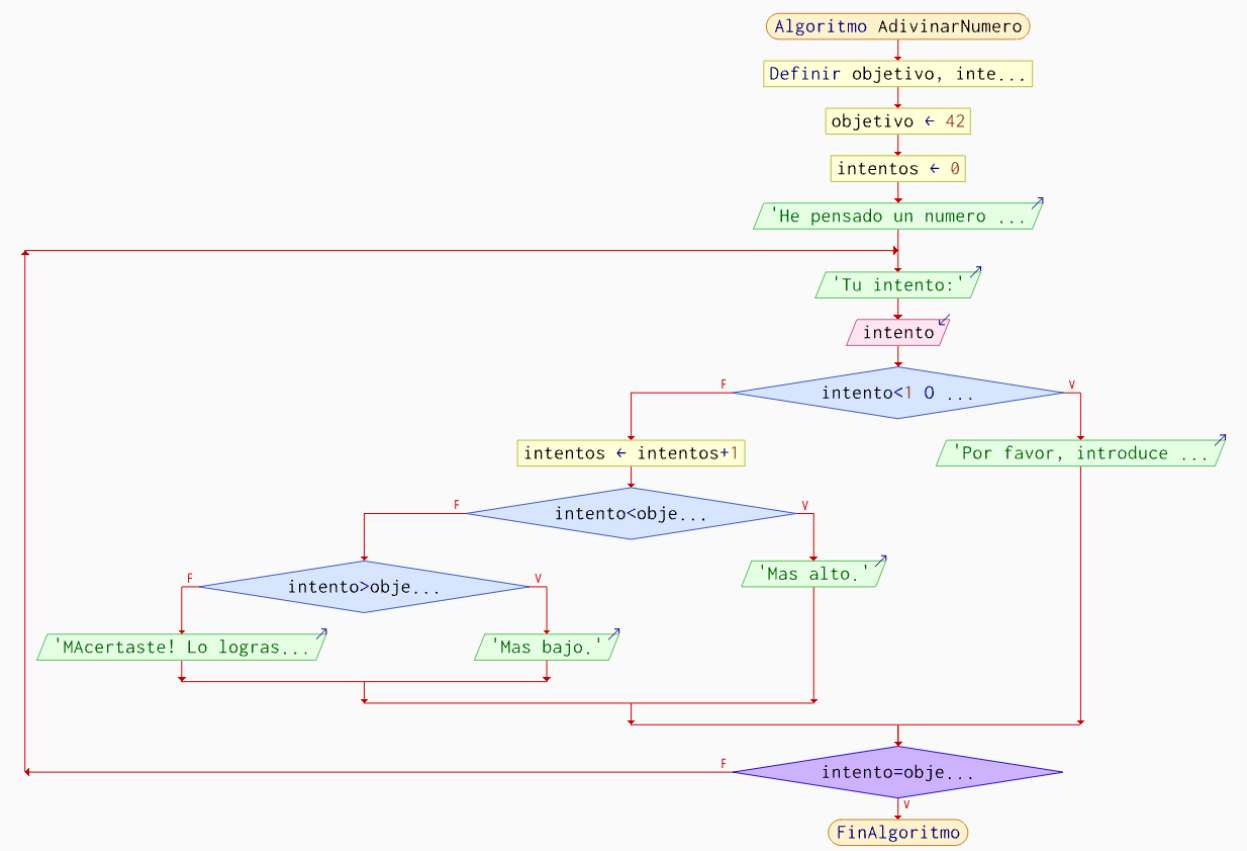
        Repetir
            Escribir "Tu intento:";
            Leer intento;

            Si intento < 1 O intento > 100 Entonces
                Escribir "Por favor, introduce un numero entre 1 y 100.";
            SiNo
                intentos <- intentos + 1;

                Si intento < objetivo Entonces
                    Escribir "Mas alto.";
                SiNo
                    Si intento > objetivo Entonces
                        Escribir "Mas bajo.";
                    SiNo
                        Escribir "¡Acertaste! Lo lograste en ", intentos, "
intentos.";
                    FinSi
                FinSi
            FinSi
        Hasta Que intento = objetivo
```

FinAlgoritmo

Diagrama de flujo



Prueba de escritorio

PASO	intento	intentos	SALIDA
1	98	1	“Mas bajo”
2	12	2	“Mas alto”
3	35	3	“Mas alto”
4	67	4	“Mas bajo”
5	44	5	“Mas bajo”
6	42	6	“Acertaste lo lograste en 6 intentos ”

Ejecutado Codeblocks

```c
#include <stdio.h>

//Ejercicio20 adivino numero
int main(void) {
    int objetivo, intento;
    int intentos = 0;

       objetivo = 42;


    printf("He pensado un numero entre 1 y 100. ¡Adivinalo!\n");

    while (1) {
        printf("Tu intento: ");
        if (scanf("%d", &intento) != 1) {
            printf("Error: entrada invalida.\n");
            while (getchar() != '\n');
            continue;
        }

        if (intento < 1 || intento > 100) {
            printf("Por favor, introduce un numero entre 1 y 100.\n");
            continue;
        }

        intentos++;

        if (intento < objetivo) {
            printf("Mas alto.\n");
        } else if (intento > objetivo) {
            printf("Mas bajo.\n");
        } else {
            printf("¡Acertaste! Lo lograste en %d intentos.\n", intento
```

```
"C:\Users\sabes\OneDrive\De
He pensado un numero entre 1 y 100. íAdivinalo!
Tu intento: 98
Mas bajo.
Tu intento: 12
Mas alto.
Tu intento: 35
Mas alto.
Tu intento: 67
Mas bajo.
Tu intento: 44
Mas bajo.
Tu intento: 42
íAcertaste! Lo lograste en 6 intentos.

Process returned 0 (0x0)   execution time : 24.795 s
Press any key to continue.
```